# Entropy-based Fuzzy Deduplication with Perfect Resistance to Key Recovery Attack

Zehui Tang
*School of College of Computer Science and Technology*
*Nanjing University of Aeronautics and Astronautics*
Jiangsu, P.R.China

Shengke Zeng*
*School of Computer and Software Engineering*
*Xihua University*
Chengdu, P.R.China
Corresponding author

Minfeng Shao
*School of Computer and Software Engineering*
*Xihua University*
Chengdu, P.R.China

***Abstract*—Deduplication for encrypted data reduces the storage costs of server while keeping the sensitive data secure. Fuzzy deduplication is developing for the multimedia data for its similarity not exact equality, hence it has better storage space saving capabilities than exact deduplication. However, the most of existing works face the security threat such as brute-force guessing attacks, key recovery attacks and so on. We conduct research on fuzzy deduplication from the fundamental layer for these security goals. In this work, the data similarity is novelty defined based on information entropy theory so that the similarity verification by cloud is without online clients assistance. Moreover, the parameter robustness verification is firstly proposed to be against the key recovery attacks and guessing attacks fundamentally. The simulation experiments show that our scheme can save approximately 87% of storage space while maintaining a tamper detection success rate of no less than 97%.**

***Index Terms*—Fuzzy Deduplication, Information Entropy, Cloud Storage, Data Security, Non-interactive Key Transfer.**

## I. Introduction

Cloud storage is an online accessible storage model that stores data in a cluster of remote servers. [1] states that 86% of respondents choose cloud storage to save costs. With the rapid growth in data user outsourcing, cloud storage service providers (CSPs) tend to *deduplication technology* to save on storage costs. If multiple clients upload the same/similar files, the storage server detects the duplicates and retains only the unique copy. Past studies have shown that deduplication can effectively save 50% of storage space [2] (up to 98% [3] of storage space for backup systems), which has driven the adoption of deduplication in many cloud storage service providers (e.g. Google Drive, Dropbox, Alibaba Cloud, Bitcasa, Mozy and Memopal) to utilize deduplication technology to save on storage costs [4]. In order to protect the privacy of data, customers using cloud storage prefer to encrypt or hide data on the client side using encryption [5] or information hiding [6] techniques. However, simple encryption hinders deduplication because the same data can be uploaded as completely separate ciphertexts due to differences in the encryption keys chosen by the client. Message protection techniques based on information hiding increase the difficulty of deduplication even more because the storage server cannot detect the hidden information effectively.

Depending on the accuracy of deleted data, related works can be divided into two categories: 1) Deduplication for the Same Data [7]–[13]. The current mature solution is to use Message Locked Encryption (MLE) [7]–[10], [14] or a secondary server-assisted MLE (SMLE) [10]–[12]. The encryption keys of MLE and SMLE originate from the data content, a kind of deterministic encryption susceptible to offline brute-force attacks. In addition, SMLE is a strong assumption (i.e., the secondary server is honest and trustworthy) and is susceptible to conspiracy and online brute-force attacks. In addition, these schemes do not support similar data deduplication, as plaintext with minor gaps will be encrypted into a completely different ciphertext. 2) Deduplication for Similar Data. Similar data deduplication is mainly realized based on block identity [15] and perceptual features [16]–[21]. The block identity means that the file is divided into $N$ blocks, then set a verification threshold ($T$), if the number of identical blocks of two or more files is greater than or equal to $T$, then the files are considered similar. Perceptual features means extracting the summaries information from the files, and then calculating the distance between the summaries of the files (e.g., Hamming, Cosine and Jacobi distance).

Although the above schemes can achieve secure deduplication of similar data, they still have some drawbacks. [15] achieves similar data deduplication by chunking files and combining it with MLE. This enables the deduplication of fixed-size files, and the size and number of chunks are challenging to be constant in practical applications. The scheme in [16] cannot realize multi-user cross-group deduplication. [17] proposed a single server deduplication scheme to solve the almost identical image deduplication problem. The scheme is not resistant to brute-force attacks according to the argument of Takeshita et al. [19], [20] require that previous users are protected online in order to validate data with subsequent users, and the assumption that all users remain online is

unreasonable. [18], [21] based fuzzy extractor ($FE$) [22] proposes an similar deduplication scheme across users that does not require users to stay online. As stated by the proposer of $FE$, in his subsequent work [23], $FE$ has to compute the public auxiliary string $P$ in addition to the extracted key $R$, and then use $w'$ and $P$ to recover $R$. Unfortunately, once $P$ has been tampered with, it will mean that the $FE$ cannot recover the correct key $R$. Therefore, [18], [21] cannot verify the prevention of forgery attacks, while [18] also explicitly states that their scheme is not resistant to online brute-force attacks by internal adversaries. It is worth noting that although the above schemes are similar data deduplication, they have no reasonable explanation of *what kind of data can be considered as similar data?*

In order to realize secure fuzzy deduplication (i.e., deduplication of similar data) in the outsourcing field, the following issues must be addressed to meet the practical requirements. (1) What kind of data can be regarded as similar, and how can the degree of similarity be measured? [18] states that data with similar visual senses are similar. This interpretation is not perfect because some of the data is not visually meaningful and the definition does not give a tool to measure similarity. (2) The cloud service provider should check the similarity between the new data and the stored data without client assistance. As a practical consideration, the similarity check should not rely on the assistance of any online client, as it is unrealistic to require the client to be online at all times. (3) The cloud server should have the ability to discern whether the client has ownership of the stored data. [24], [25] suggest that Proof of Ownership (PoW) techniques cannot be used to verify the ownership of similar files because they prevent verifiers with similar files from passing through the verification process. [18] proposed that FuzzyPow implements PoW verification of proximate data, but FuzzyPow requires using XoR encryption to maintain data similarity. When the XoR is applied to image data, it yields only weak security. The PoW authentication proposed by [21] requires the client to download the storage file from the cloud for authentication, which will undoubtedly increase the computational overhead of the client. (4) The cloud service provider should be able to verify the correctness of the metadata required in the deduplication process, especially the public parameters. [18], [21] generate a public key based on $FE$ to assist in transmitting parameter $P$. If $P$ is tampered with, it will lead to an error when recovering the decryption key.

In this paper, we propose entropy-based fuzzy deduplication (EFDep), a system specialized in secure deduplicating similar data in non-trusted execution environments. Firstly, we define similar data from the perspective of information entropy to fill up the lack of proximity evaluation criteria in related research fields. Secondly, we design a secure fuzzy detection strategy based on entropy that enables cloud service providers to verify the similarity between hash values of new data and the stored data without online clients assistance. Thirdly, in order to avoid brute-force guessing attacks from the deterministic hash, we implement a private computation of the similarity of hash by using the short hashing technique [13] and the exchange table (ET) [19]. In addition, Robust Fuzzy Extract (RFE) and Algebraic Manipulation Detection Codes (AMD) [23] are also used for public parameters verification in the deduplication process. To the best of our knowledge, it is the first time that these ideas have been applied in deduplication schemes. Finally, we design an RFE-based fuzzy ownership verification to realize the ownership of stored similar data. Our contributions are summarized as follows and the comparison of related deduplication schemes is listed in Table I:

- We give a formal evaluation criterion for data similarity from the perspective of information entropy and, for the first time, generate experimental datasets with different degrees of similarity for the deduplication domain. Although standard algorithms for evaluation can achieve data similarity measures, they cannot measure the differences between information distributions. Similar data cannot be measured simply by visual senses but by the information entropy of whether the data are similar.
- We propose a secure private computation of the similarity of hashes. Without knowing the specific data fingerprint, the cloud service provider can still compute the similarity between the new data fingerprint and the fingerprint in the cloud server. A fuzzy ownership verification protocol is proposed that enables CSPs to verify that subsequent distributors do own similar files.
- We develop a non-interactive key security sharing mechanism based on RFE and AMD that allows legitimate clients to recover the encryption key used by the initial distributor, ensuring that they can correctly decrypt the ciphertext stored in the cloud.
- The scheme is resistant to brute-force attacks and key recovery attacks. Finally, we perform a formal security analysis and a comprehensive performance evaluation to demonstrate that our EFDep is secure and efficient (approximately 87% savings in storage and the tamper detection success rate of no less than 97%).

TABLE I
COMPARISON OF RELATED SCHEMES

| Test Group | TD | SS | FD | CR | BFAR | RAR | LC | CG | CA | NKT |
|---|---|---|---|---|---|---|---|---|---|---|
| Jiang [18] | × | × | √ | × | √ | √ | × | √ | × | √ |
| Chen [29] | × | √ | × | × | √ | × | × | × | × | × |
| Takeshita [17] | × | √ | √ | √ | × | √ | × | √ | × | × |
| Cheng [19] | × | √ | √ | √ | √ | √ | √ | √ | × | × |
| Zhou [31] | × | × | √ | × | √ | √ | × | √ | × | × |
| Tang [20] | × | √ | √ | √ | √ | √ | √ | √ | √ | × |
| Chen [15] | × | √ | √ | × | √ | × | × | √ | × | × |
| Song [21] | × | √ | √ | × | √ | × | √ | √ | × | √ |
| Zheng [13] | × | × | × | × | √ | × | × | √ | × | × |
| **Ours** | √ | √ | √ | √ | √ | √ | √ | √ | √ | √ |

$'\surd' =$ Satisfied $'\times' =$ Unsatisfied SS = Single Server
TD = Tampering Detection FD = Fuzzy Deduplication CR = Collusion Resistance BFAR = Brute-Force Attacks Resistance
RAR = Replay Attacks Resistance LC = Label Consistency CG = Cross Group
CA = Color Awareness NKT: Non-interactive Key Transfer

## II. PRELIMINARIES

### A. Kullback-Leibler

**Kullback-Leibler (KL):** KL is a measure of the asymmetry of the difference between two probability distributions, which is also known as the relative entropy. The Kullback-Leibler measures the distance between two distributions of the same random variable. The KL is computed as shown in the formula 1. where $D_{KL}(p||q))$ denotes the $KL$ distance from the distribution $q$ to the distribution $p$, $p(x_i)$ is the probability of the event $X = x_i$ under the distribution $p$, $q(x_i)$ is the probability of the event $X = x_i$ under the distribution $q$.

$$D_{KL}(p||q) = \sum_{i=1}^{n} p(x_i) \log \frac{p(x_i)}{q(x_i)} \tag{1}$$

### B. Secure Sketch and Fuzzy Extractor

**Secure Sketch [22].** Let $M$ be the metric space of the distance function $dis$. As shown below, Fig.1 depicts the syntax of Secure Sketch:

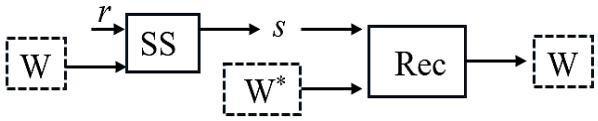


Fig. 1. Syntax Depiction of Secure Sketch (W is similar to W*)

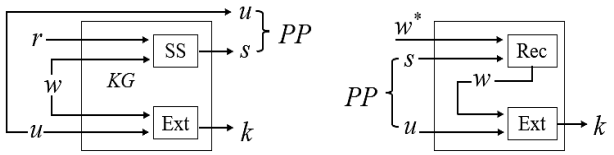


Fig. 2. Depiction of Fuzzy Extractor

**Definition 1** Secure Sketch: An (M, $\tau$, m, $\breve{m}$ , $\ell$ ) is a pair of randomized procedures, sketch (SS) and recover (Rec), with the following properties:

- **SS(w,r) $\rightarrow$ {s}:** Take as input the message $w$ and a random string $r$ and get the binary string $s$. where $w \in M$, $r \in \{(0,1)\}^{\ell}$, $s \in \{(0,1)\}^{\ell}$.
- **Rec($w^*$,s) $\rightarrow$ {w}:** The recovery process takes $w^*$ and $s$ as inputs to get $w$.

If dis(w,$w^*$) $\leq \tau$, then Rec($w^*$, SS(w)) = w. If dis(w,$w^*$) $> \tau$, then Rec($w'$, SS(w))$\neq$ w. If, for any distribution $w$ on $M$ with minimum entropy $m$, the value of $w$ can be evaluated at a rate no greater than $2^{-\breve{m}}$ (i.e., $\breve{H}(W|SS(W) \leq \breve{m})$) with the probability of observing an adversary recovery of $s$, it is considered secure.

**Fuzzy Extractor**. As depicted in Fig.2, it allows some randomness $k$ to be extracted from $w$, and then $k$ is successfully reproduced from any string $w^*$ close to $w$. The reproduction uses an auxiliary string $PP$ generated during the initial extraction. However $PP$ does not need to be kept secret, since $k$ does appear to be random even given $PP$.

**Definition 2:** Fuzzy Extract [22]: An (M,m, $\ell$, $\tau$, $\sigma$)-fuzzy extractor consists of a key generate algorithm (FKG) and a reproduce algorithm (REP), with the following properties:

- **FKG(w,r,u) $\rightarrow$ {k,PP}:** It takes as input $w \leq M$ and two random binary numbers $r, u \leq \{0,1\}^{\ell}$, and outputs a public parameter ($PP = \{u, s = SS(w,r)\}$) and the extracted private string $k = Ext(w,u)$.
- **REP($w^*$,PP) $\rightarrow$ {k}**: First enter $w' \leq M$ and the string $s$, and recover $w = Rec(w^*, s)$. Finally recover $k = Ext(w,u)$ with the assistance of the public parameter $PP$.

To emphasize: 1). The above holds if dis(w,$w^*$)$\leq\tau$. 2). For $w$ with minimum entropy of $m$, any adversary can only obtain $k$ with probability less than $2^{-(m-\sigma)}$, even if $PP$ is known.

### C. Algebraic Manipulation Detection Code

An $(S, G, \delta)$-algebraic manipulation detection code, or $(S, G, \delta) - AMD$ code [23] for short, is a probabilistic encoding map $\varepsilon : S \rightarrow G$ from a set $S$ into an (additive) group $G$, together with a (deterministic) decoding function $D : G \rightarrow S \cup \{\perp\}$ such that $D(\varepsilon(s)) = s$ with probability 1 for any $s \in S$. The security of an AMD code requires that for any $s \in S$, $\triangle \in \vartheta$, Pr$[D(\varepsilon(s) + \triangle) \notin \{s, \perp\}] \leq \delta$.

**Definition 3:** An $(n, m, K, \rho) - AMD$ code is said to be a strong $(n, m, K, \rho)$ algebraic manipulation detection $(AMD)$ code if for any $s \in S$, $\triangle \in Gn\{0\}$, the probability of $Dec(E(s) + \triangle) \in \{s, \perp\}$ is at most $\rho$, i.e.,

$$Pr(Dec(E(s) + \triangle) \notin \{s, \perp\}) \leq \rho < 1, \tag{2}$$

where $|S| = m$ and $|G| = n$.

**Definition 4:** An $(n, m, K, \rho) - AMD$ code is called a weak $(n, m, K, \rho)$ code if for any $\triangle \in Gn\{0\}$ and any random $s \in R_S$ rather than an arbitrary one, the probability

$$\sum_{s \in S} Pr(s) \sum_{g \in A_s} Pr(E(s) = g) Pr(Dec(g + \triangle)) \notin \{s, \perp\} \leq \rho < 1. \tag{3}$$

### D. Exchange Rules

The data fingerprint is directly handed over to servers or third parties, which may launch brute-force attacks against the fingerprints, leading to the disclosure of user data information. Using exchange rules, data fingerprints can be made available and invisible. The exchange rule consists of three components: exchange coefficients ($ex$), combination functions ($combine()$), and exchange table ($ET$).

**Definition 5 [19]:** As shown in Fig.3, one party ($C$) in the protocol offers to provide an odd-sign exchange factor $ex_1$ , and the other party ($S$) offers an even-sign exchange factor $ex_2$. When the parties receive $ex$ from each other, they rearrange the positions by using the exchange function combine to generate the exchange table $ET$. $C$ and $S$ will be given private $ET$, called $ET_C$ and $ET_S$, respectively. we use the term $ET(str)$ to denote the operation of exchanging the string $str$ with the table $ET$ with the following attributes.

- **Comparability:** for a string $str$ and two exchange tables $ET_C$ and $ET_S$ generated by the same set of exchange factors, there is $ET_C(ET_S(str) = ET_S(ET_C(str))$

- **Reversibility:** for a string $str$ and exchange tables $ET_C$ , there is $ET_C(ET_C(str) = str$

### *E. System Model and Definitions*

Fig. 4 shows the model of EFDep, which consists of three types of entities: Primary Client (C), Cloud Server (S) and Secondary Client ($SC_i$). The client who uploads data to the S for the first time is called the C, and the client who has the same/similarity with the C is called the $SC_i$.

- (Primary Client - C ): C outsources data to S storage. To protect data confidentiality, C needs to encrypt the data before outsourcing it. In addition, it needs to generate data fingerprints and send them to S for similarity recognition instead of ciphertext data.
- (Cloud Server - S ): S stores C's private data. S needs to deduplicate the data to save on storage costs. S will strictly enforce the instructions but may also try to second guess client privacy or delete nonpopular data to relieve storage pressure (honestly but curiously).
- (Secondary Client - $SC_i$ ): Each $SC_i$ needs to pass PoW authentication before it can obtain the auxiliary parameters for recovering the decryption key from S.

EFDep contains 11 main algorithms ( **Setup**, **TagGen**, **SHash**, **ReTagGen**, **FuDep**, **Encrypt**, **Decrypt**, **ET.Exchange**, **FPow.Proof**, **AMD.Code**, **Recover.Key** ). The relevant definitions are given below:

- **Setup**($1^\lambda$) $\rightarrow$ $\{R_1$, Fuzzy-Database$\}$. Taking the security parameter $1^\lambda$ as input and output public random number $R_1$ and fuzzy storage structures.
- **TagGen**(M, $R_1$) $\rightarrow \{Tag_1\}$. Taking plaintext data $M$ and $R_1$ as input and output data fingerprint $Tag_1$.
- **SHash**($Tag_1$) $\rightarrow$ $\{Tag_2\}$. Taking $Tag_1$ as input and output the data fingerprint $Tag_2$ with strong collision.
- **ReTagGen**($Tag_1, R_1, R_2/R_3$) $\rightarrow$ $\{Tag_3\}$. Take $Tag_1$ and random numbers $R_1, R_2/R_3$ as input and output data fingerprint $Tag_3$.
- **FuDep**($\mathbf{R^1_{mac}}, \tau$) $\rightarrow\{(1/0)\}$. Taking public parameter $R^1_{mac}$ and similarity verification threshold $\tau_2$ as input. If S stores a similar file $M'$ satisfying $\tau$, output $1 \rightarrow R^{1'}_{mac}$, and ask SC to execute $FPow.Proof$. If no $M'$ satisfying $\tau_2$ exists, output $0 \rightarrow Encrypt$ and ask the uploading user to perform $Encrypt$.
- **Encrypt**($M, R^1_{mac}, R_m, R_3$) $\rightarrow$ $\{HR^2_{mac}, C_M\}$. Taking $M$, the public parameter $R^1_{mac}$ and the random numbers $R_m, R_3$ as inputs, it outputs the ciphertext data $C_M$ and the key recovery auxiliary parameter $HR^2_{mac}$.
- **Decrypt**($C_M, HR^2_{mac}, R^1_{mac}$) $\rightarrow$ $M$. Taking ciphertext $C_M$, $HR^2_{mac}, R^1_{mac}$ as input and output plaintext data $M$.
- **ET.EXchange**($ex_C, ex_S, Phash_C, Phash_S$) $\rightarrow$ $ET_C(ET_S(Phash_S)), ET_S(ET_C(Phash_C))$. Taking the exchange factors $ex_C, ex_S$ of C and S and the data fingerprints $Phash_C$, $Phash_S$ as inputs, the data fingerprints $ET_C(ET_S(Phash_S)), ET_S(ET_C(Phash_C))$ are outputted after double blinding.
- **FPow.Proof**($SE_{fk}(ET_Ci), HR_{mac}, R^1_{mac}$) $\rightarrow 1/0$. Taking $SE_{fk}(ET_Ci)$ encrypted by SC with $fk$ key and $HR_{mac}, R^1_{mac}$ as input and output the ownership result.
- **AMD.Code**($\{0,1\}^l$) $\rightarrow (R_{out}, R_{mac})$. Taking the binary string $\{0,1\}^l$ as input, output the AMD-code private secret $R_{out}$ and public auxiliary parameter $R_{mac}$.
- **Recover.Key**($R^1_{mac}, HR^2_{mac}$) $\rightarrow$ $K_W$. Using the auxiliary parameter $R^1_{mac}$ generated by C via $AMD$ and the auxiliary parameter $HR^2_{mac}$ provided by S as input, the decryption key $K_W$ is generated using the *Fuzzy Extractor*.

### *F. Security Model*

**Ideal Function.** We define the ideal function $F_{Ideal-Fuzzydedup}$ for fuzzy deduplication of encrypted data in Fig. 5. The main focus here is to describe the three types of participating entities in the system (the storage center S, the client C who tries to upload a file, and the existing client $SC_i$, who has already uploaded a file). A deduplication protocol is secure if it satisfies $F_{Ideal-Fuzzydedup}$. Specifically, the protocol does not reveal anything about the confidentiality of C's files and keys, and only $S$ knows whether the deduplication command was executed. In addition, we require that the protocol is secure in the malicious model [27], where the participants can execute arbitrary commands (refusal to participate in the protocol, input with forged messages, early termination of the protocol is allowed)

**Threat Model**. A malicious adversary $\mathcal{A}$ may compromise any of $C, S$ and $SC_i$, or conspire with $C, S$ and $SC_i$ to capture the security of the individual file uploading process by requiring the execution of the protocol ideal model $F_{Ideal-Fuzzydedup}$. In addition, all three entities in the system can potentially be called malicious adversaries, and we characterize the potential harm posed by each entity based on the characteristics of each entity, as follows.

- **Online Brute-force Attack:** For predictable files (e.g., clinical care, smart grid, etc.), the uploader can construct all the candidate files corresponding to the predicted file, upload them and observe which one leads to deduplication.
- **Offline Brute-force Attack:** If S is given a deterministic representation of the stored data (e.g., cryptographic hashing or convergent encryption), then S can construct all candidate files and verify them offline.
- **Fake Attack:** S can be faked as C by running $F_{Ideal-Fuzzydedup}$ on each guessed data to check if deduplication has occurred.

In addition to brute-force attacks, a compromised S can easily detect whether a file is in storage by running a deduplication protocol. We claim that there is no known deduplication scheme that prevents this attack because S always knows that deduplication occurs. Such attacks are not in our threat model.

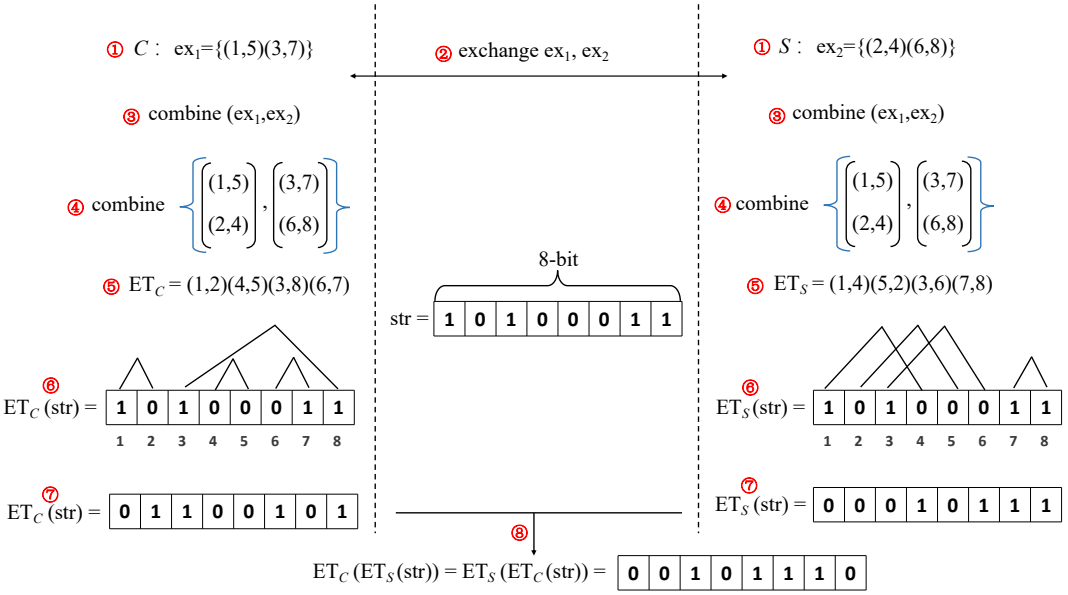


Fig. 3. 8-bit Binary Sequence Convert under Exchange Ruler

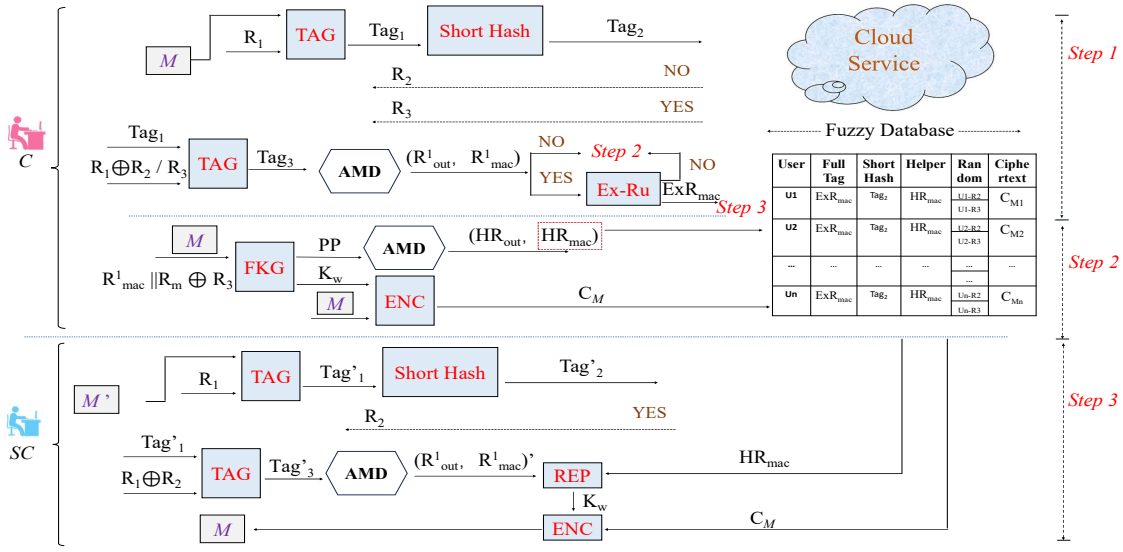


Fig. 4. System Workflow of EFDep

### G. Design Goals

In this study, design goals are discussed in terms of safety and functionality.

**Security Goals.** Security goals include correctness and privacy. Correctness requires that redundant data can be deduplicated and that legitimate clients can recover files correctly. First, for any two clients with similar plaintexts (e.g., $M$ and $M'$) that wish to store their ciphertexts (i.e., $C_M$ and $C_{M'}$) to $S$, $S$ can use a similarity checking module to detect similarity and store only one copy ($M$ or $M'$). When a deduplicated client needs to gain access to $M$ or $M'$, she/he first needs to pass the ownership proof, and then she/he can recover the decryption key in a valid time. Second, during the data ownership proof process, the output of the deduplicated client (i.e., the Hamming distance of the output of $S$) must be consistent with the actual inputs of $C_i$ and $S$. Privacy implies being available but invisible to the data. When the message is unpredictable (i.e., high information entropy) and the hash function of similarity has both the summary similarity and unidirectional properties, it is difficult for an adversary (including $S$) to obtain the plaintext message. When the message is predictable (i.e., low information entropy), the hash function of similarity possesses three properties: abstract similarity, unidirectionality and conflict.

**Functional Goals.** Designed to provide secure deduplication of similar media data. The cloud is capable of performing deduplication on similar data encrypted by different clients. In addition, non-interactive key sharing should not rely on the

**Inputs:**

- The uploader $C$ has input file $F/M$;
- Each existing client $C_i$ has input $F_i$ and $K_{F_i/M_i}$;
- $S$'s input is empty;

**Outputs:**

- $C$ gets an encryption key $K_{F_i/M_i}$. If $F/M$ is similar or identical to existing file $F_i/M_i$ then $K_{F/M}$=$K_{F_i/M_i}$. Otherwise $K_{F/M}$ is random;
- Each $C_i$'s is empty ;
- $S$ gets the fingerprint $Phash(K_F/K_M, F/M)$. If there is a fingerprint $Phash(K_{F_j/M_j}/, F_j/M_j)$ is similar or identical to $Phash(K_F/K_M, F/M)$ in its storage, it learns $j$ as well. Otherwise, $S$ learns that $F/M$ is not in the storage.

Fig. 5. The Ideal Functionality $F_{Ideal-Fuzzydedup}$ of Fuzzy Deduplicating Encrypted Data. $F$: the same data; $M$: the similar data.

assistance of any third-party online entity.

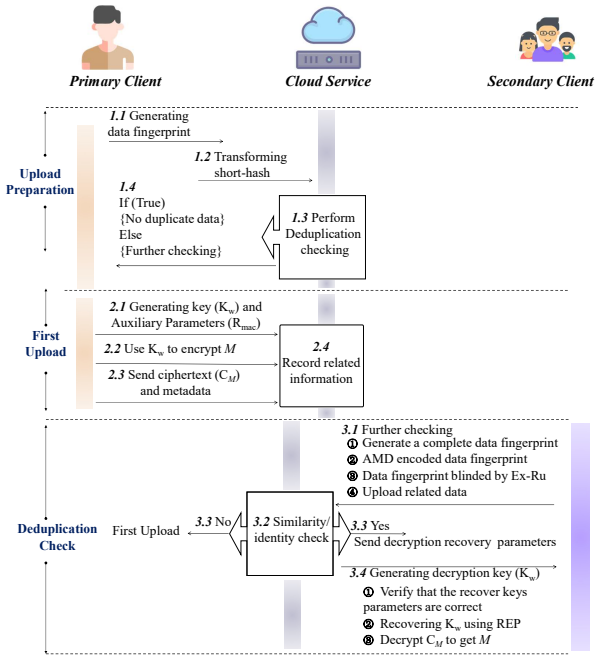


Fig. 6. General Overview of EFDep

## III. Our Constructions

### A. Overview

Before describing the construction details of the scheme in detail, we first describe the motivation for our EFDep.

When $SC$ needs to upload a file $(F/M)$ to $S$, it needs to focus on two points (i.e., existence detection and security). (1) Existence detection can determine whether an encrypted copy of $F/M$ is stored in $S$. (2) Security can be described as: existence check is true, i.e., performing a PoW test on $SC$ is safe, and $S$ sends the auxiliary metadata for recovering the encryption key to $SC$ after the successful test.

In traditional client-based deduplication, in order to save bandwidth consumption, $C$ first sends $F/M$ cryptographic hashes (e.g., SHA 256) $h = hash(F/M)$ to S to check whether S has stored $F/M$. Since the cryptographic hash algorithm is a linear mapping, a semi-honest S can easily launch an offline brute-force guessing attack on the hash value for predictable files (e.g., medical data). To defend against brute-force attacks, bilinear pairwise operations are introduced. $C$ and $C_i$ generate random numbers $r1, ri$ respectively and exchange $g^{r1}, g^{ri}$ with each other, and finally $C$ sends $e(M_1^{r1}, g^{ri})$ to $S$. Similarly $C_i$ sends $e(M_i^{ri}, g^{r1})$ to $S$. $S$ verifies that $e(M_1^{r1}, g^{ri}) \stackrel{?}{=} e(M_i^{ri}, g^{r1})$, if they are equal then the replica file $F/M$ is stored in $S$, and vice versa, no replica file $F/M$ is stored. Based on bilinear pairs, although it can resist brute-force attack, it requires $C_i$ to be always online. It cannot realize similar data detection because bilinear pairs operation is an exact operation that does not support approximation operation. Therefore, we use short hash $sh = SH(F/M)$ instead of $h$ to check rough copy. Since $SH()$ is strongly collisional, $S$ cannot accurately guess the contents of $F/M$ offline based on $sh$.

Although the short hash can resist the brute-force attack by $S$, its collision also leads to the similarity/same identification error. It is worth noting that the execution result of the short hash is false, which means that there must be no copy of $F/M$ stored in $S$, and the execution result is true, which means that there may be a copy of $F/M$ stored in $S$. Therefore, in our protocol, short hash only plays the role of pre-verified duplicate data. When the execution result is true, $C$ first encodes the complete data fingerprint by $AMD$ to get $R_{out}, R_{mac}$, and then transmits $R_{mac}$ to $S$ after Ex-Ru blinding, and then $S$ verifies the blinded fingerprints to be the same as the one stored in the database. Then $S$ verifies the distance between the blinded $R_{mac}$ and the Hamming distance of the fingerprint stored in the database to verify the existence of a copy of $F/M$. There are still a couple of issues that need to be addressed:

1). *How to prevent uploaders from learning about stored files?* EFDep supports client-side deduplication, so the client $C$ can know whether the data to be uploaded is stored in the database. To the best of our knowledge, there is no protocol that resists such attacks. Therefore, we focus on preventing malicious $C$ from launching online brute-force attacks via data fingerprinting. To solve this problem, we set up three policies (short hash, AMD encoding, and exchange rule), see Fig. 4.

2). *How can compromised $S$ be prevented from launching brute-force attacks (online and offline)?* For predictable files, $S$ mainly launches offline brute-force attacks by matching the data fingerprints of all predicted files with the data fingerprints uploaded by $C_i$. For non-predictable files, $S$ mainly launches online brute-force attacks by constantly interacting with $C/C_i$ to recognize $F/M, F_i/M_i$. To address offline attacks, we require the data fingerprints to be random, i.e., the data fingerprints are randomly blinded and then stored to $S$. To address online attacks, we require limiting the number of communications between $C/C_i$ and $S$ (specifically, $C/C_i$ will set a limit on the number of responses, and if the limit is

**Algorithm 1:** Robust Fuzzy Extractor Algorithm

**Init()** outputs a random seed $i$ for the extractor $Ext$ as a shared *common reference string*.

**Gen**($w$, $i$) does the following:

- $R \leftarrow Ext(w, i)$ which we parse as $R = (R_{mac}, R_{out})$.
- $s \leftarrow SS(w)$, $\sigma \leftarrow MAC(s, R_{mac})$, $P := (s, \sigma)$.
- Output $(P, R_{out})$.

**Rep**($w'$, $P^*$, $i$) does the following:

- Parse $P^* = (s^*, \sigma^*)$. Let $w^* \leftarrow Rec(w', s^*)$. If $d(w^*, w') > t$ then output $\perp$.
- Using $w^*$ and $i$, compute $R^*$ and parse it as $R^*_{out}, R^*_{mac}$.
- Verify $\sigma^* = MAC(s^*, R^*_{mac})$. If equation holds output $R_{out}$, otherwise output $\perp$.

exceeded within a certain period, it will not interact with $S$).

3). *How to prevent critical data from being tampered with?* In client-side deduplication protocols, security problems (e.g., data forgery, malicious backdoor injection, etc.) will arise if critical data such as data fingerprints and stored ciphertexts are tampered with. In order to solve this problem, we propose to use a robust fuzzy extractor to realize the detection of tampered data. A robust fuzzy extractor (see Aigorithm 1) to achieve listening approximation data tampering detection with non-interactive key transfer implementation principle is shown in Fig. 7.

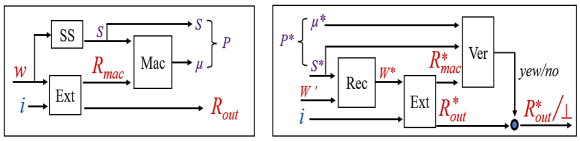


Fig. 7. Construction of Robust Fuzzy Extractor.

This paper aims to implement safe deduplication and fuzzy deduplication. We briefly explain as shown in Fig. 6. First (*Upload Preparation*), $C$ needs to generate the data fingerprint $Phash$ locally, and then transform the fingerprint into a short hash $sh$ and send it to $S$, which retrieves whether $sh$ is stored or not and sends further operation commands (i.e., deduplication or further verification) to $C$. Second (*First Upload*), $C$ generates the encryption key $K_w$ and the key recovery parameter $R_{mac}$ and sends $\{C_w, R_{mac}\}$ to $S$. Third (*Deduplication Check*), $C$ generates blinded fingerprints and sends them to $S$, $S$ calculates the Hamming distance ($Ham(Phash_c, Phash_{sc_i})$) between fingerprints, if the Hamming distance is less than the threshold value $\tau$, then $S$ sends the key recovery parameter to $C$, and $C$ generates the key $K_w$ using the REP. If the Hamming distance exceeds the threshold, $S$ informs $C$ to perform the *First Upload*. Table IV (refer to appendix) provides an explanation of important notations.

### B. Upload Preparation

**Initialization**: $S$ executes **Setup**($1^\lambda$) $\rightarrow$ {$R_1$, Fuzzy-Database} and sends $R_1$ to the legitimate users in the system. [1]

- $C$ performs **TagGen**(M, $R_1$) $\rightarrow \{Tag_1\}$ to obtain the deterministic fingerprint $Tag_1$ of $F/M$. The detailed construction of the TagGen algorithm is shown in Algorithm 2.
- $C$ executes **SHash**($Tag_1$) $\rightarrow \{Tag_2\}$ generate a collision hash $Tag_2$ (i.e., a short hash $sh$) of $Tag_1$ and sends $Tag_2$ to $S$.
- $S$ performs $d_1 = Ham(Tag_2^c, Tag_2^{ci})$. If $d_1$ is less than the threshold $\tau_1$ then $C$ is asked to perform further validation, and if $d_1$ is greater than the threshold $\tau_1$ then $C$ is asked to upload the encrypted file (i.e., perform the *First Upload*).

**Algorithm 2:** Perceived hash of $F/M$

**Input** : $F/M$
**Output** : n-bit hash value of $F/M$, $\{0,1\}^n$
Data converted to frequency domain data. // DCT, DFT, etc.
Setting the high-frequency discard threshold $\tau$.
Frequency components below $\tau$ are set to 0.
Frequency data converted to spatial data.
Spatial data is divided into $n$ blocks.
Calculating information entropy for n blocks $H(x) = -\sum_{i=1}^{n} p(x_i) \log(p(x_i))$.
calculating the mean value of information entropy $Avh(X) = \frac{1}{N} \sum H(X_i)$.
Normalization calculation: greater than or equal to $Avh(X)$ is set to 1, less than to 0.

### C. First Upload

The fact that $C$ performs a *First Upload* means that there is no copy file of $F/M$ stored in $S$.

- $C$ executes **ReTagGen**($Tag_1, R_1, R_3$) $\rightarrow \{Tag_3\}$ generate $F/M$ full-length fingerprint $Tag_3$. $Tag_3$ = $Tag_1 \oplus R_1 \oplus R_3$, where $R_3$ is derived from $S$.
- $C$ executes **AMD.Code**($\{0,1\}^l$) $\rightarrow (R^c_{out}, R^c_{mac})$. Detailed construction of the AMD.Code algorithm, see Algorithm 3.
- $C$ and $S$ negotiate a set of exchange factors, where $C$ generates exchange factors $ex_C$ for odd bits and $S$ generates exchange factors $ex_S$ for even bits.
- As shown in Fig. 3, $C$ unions $ex_C$ and $ex_S$ to randomly generate $C$'s private exchange rule $ET_C$. Similarly, $S$ generates the private exchange rule $ET_S$.
- $C$ randomly selects $M/F$ block file $M_{bl}/F_{bl}$, and generates its corresponding fingerprint $p_{bl} = Phash(M_{bl}/F_{bl})$, the corresponding serial number of the block file is $IDX$.
- $C$ generates the randomness fingerprint $ExR_{mac} = ET_C(R^1_{mac}||p_{bl})$ for $F/M$ using $ET_C$ blinding $R^1_{mac}||p_{bl}$.

[1]Ignore the user registration operation here, as it's not our focus.

- $C$ Execute **Encrypt**$(M, R^1_{mac}, R_m, R_3) \rightarrow \{HR_{mac}, C_M\}$ algorithm to generate the ciphertext of $F/M$, $C_M$, and the parameter of encryption key recovery, $HR_{mac}$. Detailed construction of the Encrypt algorithm is described in Algorithm 4.
- As shown in Fig. 7, $C$ uses Fuzzy Extractor to generate a fuzzy key ($fk$), and uses $fk$ to encrypt $ET_C$ to get the ciphertext of the blinding rule $C_{ET} = SE_{fk}(ET_C)$.
- $C$ sends $\{C_M, ExR_{mac}, HP_{mac}, C_{ET_c}, IDX\}$ to $S$.

**Algorithm 3:** AMD.Code

**Input** : The file $S \in \mathbb{F}_2^l$, and random coefficient $R \in \mathbb{F}_2^\tau$, where $\tau \mid l$.

Rewrite $S$ as

$$S = (S_1, S_2, \ldots, S_{\frac{l}{\tau}}),$$

where $S_i \in \mathbb{F}_2^\tau$ can be regard as an element of $\mathbb{F}_{2^\tau}$.

Define

$$f(x) \triangleq x^{\frac{l}{\tau}+2} + \sum_{1 \le j \le \frac{l}{\tau}} S_j x^j \in \mathbb{F}_{2^\tau}[x].$$

**Output** : $g = (S, f(R)) \in \mathbb{F}_2^{l+\tau}$.

An AMD Code with parameters $(2^{l+\tau}, 2^l, \frac{\frac{l}{\tau}+1}{2^\tau})$.

**Algorithm 4:** Encryption algorithm

**Input** :$M, R^1_{mac}, R_m, R_3$.

**Output** : $HR_{mac}, C_M$.

$C$ takes $M$ and $R^1_{mac}||R_m \oplus R_3$ as inputs to the FKG, and then obtains the private key $K_w$ and the publicizable parameter $PP$.

$C$ takes $PP$ as input for AMD coding to get auxiliary parameters $HR_{out}, HR_{mac}$.

$C$ encrypts $M$ using $K_w$, $C_M = SE_{K_w}(M)$.

$C$ sends $HR_{mac}, C_M$ to $S$.

### D. Deduplication Check

Performing the *Deduplication Check* implies that a copy of $F/M$ may be stored in $S$. Therefore, it is necessary to further confirm whether a copy of $F/M$ is stored in $S$, and $S$ also needs to perform PoW verification on $C$.

- $S$ sends $C_{ET_{Ci}}$ and $IDX$ of candidate $SC_i$ to $C$.
- $C$ generates $fk$ via Fuzzy Extractor and $p_{bl}$ via $Phash$.
- $C$ decrypts $C_{ET_{Ci}}$ using $fk$ to get $ET_{Ci}$.
- $C$ blinds the data fingerprint using $ET_{Ci}$, $ET_{Ci}(R^1_{mac}||p_{bl})$.
- $C$ and $S$ perform a new round of "exchange rules" to blind the data fingerprints and measure the similarity between them, see Algorithm 5.
- If $d_2$ is less than the threshold $\tau_2$ then $S$ informs $C$ that there is no need to upload data and transforms the identity of $C$ to $SC_i$. $S$ sends $HP_{mac}$, an auxiliary parameter for recovering $K_W$, to $C$. $C$ recovers $K_W$ using *Fuzzy Extractor*.
- If $d_2$ is greater than the threshold $\tau_2$ then $S$ notifies $C$ to perform a *First Upload*.

**Algorithm 5:** $S$ securely verifies that $C$ has $F/M$ copy files

**Input** : $str_C = \{0,1\}^n, str_S = \{0,1\}^n$

**Output** : 1/0 // 1: copy of $F/M$ exists; 0: no copy of $F/M$.

$C$ and $S$ negotiating new round of $ex_C, ex_S$. $S$ and $C$ perform combine( $ex_C, ex_S$) to get $ET'_C, ET'_S$.

$S$ executes $ET_S(SM)$=$ET'_S(ET_{Ci}(R^1_{mac}||p_{bl}))$, and sends $ET_S(SM)$ to $C$.

$C$ performs $ET_C(CM)$=$ET'_C(ET_{Ci}(R^1_{mac}{}'||p_{bl}))$, and sends $ET_C(CM)$ to $S$.

$S$ executes $Phash_S$=$ET'_S(ET_C(CM))$, and sends $Phash_S$ to $C$.

$C$ executes $Phash_C$=$ET'_C(ET_S(SM))$, and sends $Phash_C$ to $S$.

$S$ and $C$ executes $d_2 = Ham(Phash_S, Phash_C)$.

If $d_2 \le \tau_2$ outputs 1, otherwise outputs 0.

## IV. Functionality and Security Analysis

This section analyzes the functionality and security of the deduplication system.

*Deduplication feasibility analysis:* Our goal is to achieve similar data deduplication, which implies the need for uni-directional generation of perceptual features (i.e., fingerprints that maintain similarity) from $F/M$. Ensuring that perceptual features generated from $F/M$ can be used instead of $F/M$ for similarity validation requires answering an interesting question: What kind of data is similar anymore? [18] describes similarity as data that look similar to the human eye, and such a definition lacks rigor. Therefore, we introduce KL distance to answer this question. Specifically, we give a standardized degree of similarity between data by measuring the difference in distribution between the information contained in the data (i.e., $0 \leftarrow D_{KL}(M_1||M_2)$). If $Ham(M_1, M_2) \ge \tau_2$, but $D_{KL}(M_1||M_2) \le \tau_2$, it means that the deduplication system has misjudged.

*Security analysis:* An adversary can directly attack the ciphertext and analyze the metadata to obtain valid plaintext information. It is worth emphasizing that our scheme uses a symmetric encryption algorithm to encrypt the file $F/M$, so as long as the key is not disclosed, the adversary cannot learn $F/M$ from $C_{F/M}$. Therefore, we mainly analyze the attack launched by the adversary through metadata.

This scheme aims to achieve (1) PRV-CDA security [7] (data privacy under selective distribution attacks, where the encryption of two unpredictable messages should be indistinguishable.) (2) AMD Anti-Tampering, Robust Authentication of Public Parameters (3) Ownership security under hash attacks

| **(a). PRV-CDA**$^{A}_{M}(\lambda)$ | **(b). TDSG:**$Game^{AMD_{Tag}}_{A}$ |
|---|---|
| $P \leftarrow \Re(1^{\lambda})$<br>$b \leftarrow \{0,1\}$<br>$(M_0, M_1, Z) \leftarrow M(1^{\lambda})$<br>For $i = 1, \ldots, \lvert M_b \rvert$ do<br>$C[i] \leftarrow En(Key(M_b[i]), M_b[i])$<br>$b' \leftarrow A(P, C, Z)$<br>$Ret(b = b')$ | $(P, \delta) \leftarrow \Re(1^{\lambda})$<br>$(P_A, \delta_A) \leftarrow \Re(1^{\lambda})$<br>$P_A + \triangle = (P + \triangle_1, \delta + \triangle_2)$<br>$If\ dis(M, M') \leq \tau_2$<br>$M + \tilde{\triangle} \leftarrow Rec(P + \triangle_1, M')$<br>$R_{mac} + \triangle_3 \leftarrow Ext(M + \tilde{\triangle})$<br>$If\ AMD(\delta + \triangle_3, P + \triangle_1, R_{mac} + \triangle_3)\ True$<br>$Ret\ R_{out}$<br>$Else\ Ret\ 0$<br>$Else\ Ret\ 0$ |
| **(c). OSG:**$Game^{PoW}_{A}$ | **(d). TCG:**$Game^{TC}_{A}$ |
| $fk \leftarrow FKG(R^1_{mac}, R_m, R_3))$<br>$C_{ET} \leftarrow SE^{+}_{fk}(ET_C)$<br>$fk^A \leftarrow FKG(R^1_{mac}, HR_{mac}))$<br>$ET_{Ci} \leftarrow SE^{-}_{fk^A}(C_{ET})$<br>$Phash_C, Phash_S \leftarrow ET.EXchange(\bullet)$<br>$If\ Ham(Phash_C, Phash_S) \leq \tau_2$<br>$1 \leftarrow FPow.Proof(SE^{+}_{fk}(ET_{Ci}), HR_{mac}, R^1_{mac})$<br>$Ret\ 1$<br>$Else\ Ret\ 0$ | $IDX \leftarrow \{0,1\}^{\lambda}$<br>$P_{bl} = TagGen(M_{IDX}, R_1)$<br>$P^{'}_{bl} = TagGen(M^{A}_{IDX}, R_1)$<br>$If\ Ham(P_{bl}, P^{A}_{bl}) \leq \tau_2$<br>$Ret\ 1$<br>$Else\ Ret\ 0$ |

Fig. 8. Games Defining PRV-CDA; Tmpering Detection Security game(TDSG); Ownership Security game(QSG); Tag Consistency game(TCG)

(4) Tag under deduplication-faking attack (DFK)) consistency verification.

Definition 6: We say that EFDep is secure if there is no polynomial-time adversary $\mathcal{A}$ with negligible advantage $Adv^{PRV-CDA}_{A}$ to win the $PRV - CDA^{A}_{M}(\lambda)$.

- Setup: The adversary $\mathcal{A}$ sends an unpredictable description of *block-soure M* to the challenger. The challenger $\mathcal{A}$ then runs $P \leftarrow \Re(1^{\lambda})$, gets the public parameter $P$, and sends it to the opponent $\mathcal{A}$.
- Challenge: The challenger chooses $b \leftarrow \{0,1\}$ at random. If $b = 0$, then allow $M$ to be $(M_0, Z)$. Conversely, if $b = 1$, choose $M_1$ uniformly and randomly from $\{0,1\}^{|M_0|}$. Set $M = M_b$, $b$ is the number of blocks. The challenger generates a key for each block $M_i, i \in \{1, b\}$, then computes the ciphertext $C[i]$ ($C[i] \leftarrow En(Key(Mb[i]), Mb[i])$). Finally, the challenger provides public parameters $P$, ciphertext $C$ and auxiliary information $Z$ to $\mathcal{A}$.
- Output: After $\mathcal{A}$ receives $(P, C, Z)$, he outputs his guess $b^{'}$ for $b$, and if $b = b^{'}$, then $\mathcal{A}$ wins the game.

Such an adversary $\mathcal{A}$ is called $PRV - CDA^{A}_{M}(\lambda)$ and the advantage of the adversary A is defined as

$$\mathrm{Adv}^{\text{PRV-CDA}}_{A}(\lambda) = |Pr[b = b^{'}] - \frac{1}{2}|$$

Proof: The advantage of $\mathcal{A}$ comes from the encryption key $Key(M_b[i])$ corresponding to the ciphertext $C_i$. Since a fuzzy extractor generates the encryption key of standard symmetric encryption, and the key seed is randomized, and the two files $M_0, M_1$ are unpredictable messages to the adversary, $\mathcal{A}$ does not have a guessing advantage. See [23] for the security of the *Fuzzy Extractor*.

*Anti-tamper analysis*: Our EFDep tamper detection process is shown in Fig.9. The ability of AMD code to detect tampering determines the generalization ability of the fuzzy extractor. For $w$ and $w'$, if there exists $dis(w, w') < \tau_2$, then there is $Rec(w', P^*) = Rec(w', P) = w$, and thus $R_{out}$ can be recovered by $Ext(w)$. Now consider the existence of a malicious adversary that tampers with the public parameter $P$ as $\tilde{P} = (P + \Delta_1, \sigma + \Delta_2), \sigma = f(P, R_{mac})$, which is obtained

$$\begin{aligned}
\tilde{w} &= \mathrm{Rec}(w', \widetilde{P}) \\
&= \mathrm{Rec}(w, P) + \mathrm{Rec}(w', \widetilde{P}) - \mathrm{Rec}(w, P) \\
&= \mathrm{Rec}(w, P) + \mathrm{Rec}(w', P + \Delta_1) - \mathrm{Rec}(w, P) \\
&= w + h(w' - w, P + \Delta_1, P) \\
&= w + \widetilde{\Delta}
\end{aligned}$$

Here, $\widetilde{\Delta} = h(w' - w, P + \Delta_1, P)$, and finally the veracity of $\widetilde{w}$ is verified by testing $\sigma + \Delta_2$.

$$\sigma + \Delta_2 = f(P + \Delta_1, \mathrm{Ext}_{mac}(\tilde{w})) = f(P + \Delta_1, \mathrm{Ext}_{mac}(w + \widetilde{\Delta}))$$

There exists a linear characterization of $Ext()$, so that there are

$$\begin{aligned}
\sigma + \Delta_2 &= f(P + \Delta_1, \mathrm{Ext}_{mac}(w + \tilde{\Delta})) \\
&= f(P + \Delta_1, \mathrm{Ext}_{mac}(w) + \mathrm{Ext}_{mac}(\tilde{\Delta})) \\
&= f(P + \Delta_1, \mathrm{Ext}_{mac}(w) + \Delta_3) \\
&= f(P + \Delta_1, R_{mac} + \Delta_3),
\end{aligned}$$

Here, $\text{Ext}(\tilde{w}) = (\hat{R}_{mac}, \hat{R}_{out}) \triangleq (\text{Ext}_{mac}(\tilde{w}), \text{Ext}_{out}(\tilde{w})), \Delta_3 \triangleq \text{Ext}_{mac}(\tilde{\Delta})$.

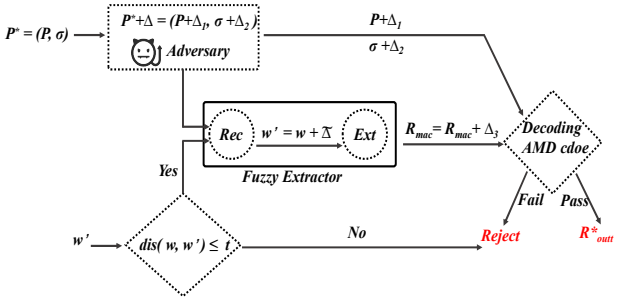


Fig. 9. Anti-tamper Attack Validation Process

Definition 7: If any probabilistic polynomial time $\mathcal{A}$ has a negligible advantage $Adv_A^{PoW}(\lambda)$ to win the game $Game_A^{PoW}$ defined by Fig. 8, then our scheme can achieve media data ownership security, where the message space $M(b)$ is large enough to make the plaintext unpredictable.

$$\text{Adv}_A^{\text{TC}}(\lambda) = |Pr[Game_A^{TC}(\lambda)] = 1|$$

Proof: The advantage of $\mathcal{A}$ comes from generating the key $fk$ to guess the file binding from the fuzzy extractor. Specifically, $\mathcal{A}$ generates $fk^A$ and then uses $fk^A$ to decrypt $C_{ET}$ to obtain $ET_{C_i}$, which is then verified with the challenger $\Delta$, i.e., satisfies $(fk = fk^A - \Delta \vee fk^A + \Delta = fk)$. The ability of $\mathcal{A}$ to recover $fk$ using the common parameter $P$ presupposes that $\mathcal{A}$ can satisfy $b = b'$ in $PRV\text{-}CDA_M^A$. Undoubtedly, $M_0, M_1$ are indistinguishable for $\mathcal{A}$. Therefore, $\mathcal{A}$ has no advantage.

Definition 8: If any probabilistic polynomial time adversary $\mathcal{A}$ has a negligible advantage $Adv_A^{TC}(\lambda)$ to win the game $Game_A^{TC}$ defined by Fig. 8 , then our scheme achieves label consistency verification.

$$\text{Adv}_A^{\text{PoW}}(\lambda) = |Pr[Game_A^{PoW}(\lambda)] = 1|$$

Proof: $\mathcal{A}$ wins the game if it correctly provides the source $M_i$ specified by the challenger. Specifically, the challenger randomly specifies $M_b$ and generates the label $P_{bl}$ from $M_i$, then lets $\mathcal{A}$ pick an $M_A$ and generate the label $P'_{bl}$ from $M_A$. Then verify that $|P_{bl} \pm P'_{bl}| \stackrel{?}{=} \Delta$. As described earlier, the message space $M(b)$ is large enough that the plaintext is unpredictable, so $\mathcal{A}$ has no advantage.

## V. Performance Analysis

Redundant dataset construction: fuzzy deduplication aims to remove similar data. However, there is a lack of approximation evaluation criteria and datasets suitable for fuzzy deduplication. Therefore, we constructed a new dataset suitable for fuzzy deduplication based on Kullback-Leibler. Specifically, we randomly selected 2000 images from the MSCOCO dataset and divided them into five groups (400 images in each group). Then, we randomly added noise to each image to form redundant copies and tested the KL distance between the noisy and the source images to evaluate the gap between the two. Table II describes the relevant details of the data set.

TABLE II
Data Set

| File Name | Interference | Number | Size |
|---|---|---|---|
| Original File ($O_1$) | - | 400 | 64.3MB |
| Copy File ($C_1$) | noise | 2000 | 440MB |
| Original File ($O_2$) | - | 400 | 62.3MB |
| Copy File ($C_2$) | noise | 2000 | 436MB |
| Original File ($O_3$) | - | 400 | 61.7MB |
| Copy File ($C_3$) | noise | 2000 | 432MB |
| Original File ($O_4$) | - | 400 | 61.8MB |
| Copy File ($C_4$) | noise | 2000 | 438MB |
| Original File ($O_5$) | - | 400 | 63.9MB |
| Copy File ($C_5$) | noise | 2000 | 439MB |
| Original File ($O_6$) | - | 2000 | 314MB |
| Copy File ($C_6$) | RGB switching | 2000 | 250MB |

TABLE III
Comparison of Computation

| Schemes | T/K-Gen | Enc | Dec | Sec-Ver |
|---|---|---|---|---|
| Jiang [18] | $Phash + 2FE$ | $SE_{Enc}$ | $SE_{Dec}$ | $N(r + OT + XoR)$ |
| Tang [20] | $2Phash + r$ | $SE_{Enc}$ | $SE_{Dec}$ | $ZK + 2XoR$ |
| Chen [15] | $(m+1)(H+E)$ | $m(H + XoR)$ | $mXoR$ | $m(P+E) + 2P$ |
| Cheng [19] | $Phash$ | $SE_{Enc}$ | $SE_{Dec}$ | $2XoR$ |
| **Our** | $Phash + 2FE$ | $SE_{Enc}$ | $SE_{Dec}$ | $2XoR + H + AMD$ |

T/K-Gen: Tag/Key Generation Enc: Encryption Dec: Decryption XoR = Exclusive OR
Sec-Ver: Secure verification H = Hash Function OT = Oblivious transfer
$SE$ = Symmetric Encryption Function P = A bilinear pairing operation
E = An exponential operation in $\mathbb{G}$

**Data tag generation time cost evaluation:** Based on the dataset of table II, the average time cost was measured by 1000 times of the Algorithm 1. As shown in Fig. 10, 64-bit tag generation take 80 seconds ($O_1$-$O_5$, 400 samples) and 410 seconds ($C_1$-$C_6/O_6$, 2000 samples), while 256-bit tags take 85 seconds and 443 seconds. The cost of single image generation is 0.2 seconds (64-bit) and 0.21 seconds (256-bit), respectively, which meets the client's need for efficient computation.

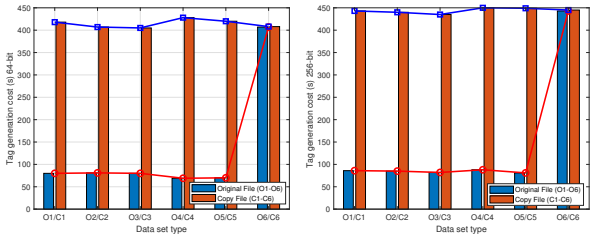


Fig. 10. Cost of Generating Data Tag on Different Datasets (left: 64-bit phash, right: 256-bit phash)

**Evaluation of encryption and decryption overhead:** Fig. 11 gives the time overhead required by four different encryption and decryption algorithms, in which AES-128 and AES-256 are significantly lower than DES and 3DES. therefore, in practical applications, we recommend using AES256 to

encrypt the data because of its strong security and less time required.

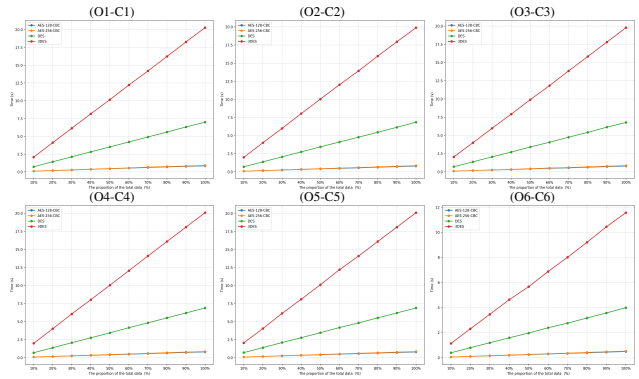


Fig. 11. Time Overhead for Different Encryption and Decryption Algorithms

**The hash length determines the collision probability:** Too short a hash significantly increases the collision probability (decreasing the recognition accuracy), while too long a hash weakens the resistance to guessing attacks. As shown in Fig. 12, we intercepted the 64-bit and 256-bit hashes in the $O_1$-$C_1$ to $O_6$-$C_6$ datasets in equal proportions (20%, 40%, 60%, 80%) and evaluated the collision probability. The experimental results show that when the short hash length is 45%-55% of the original hash length, it can achieve the optimal balance between recognition accuracy and attack resistance.

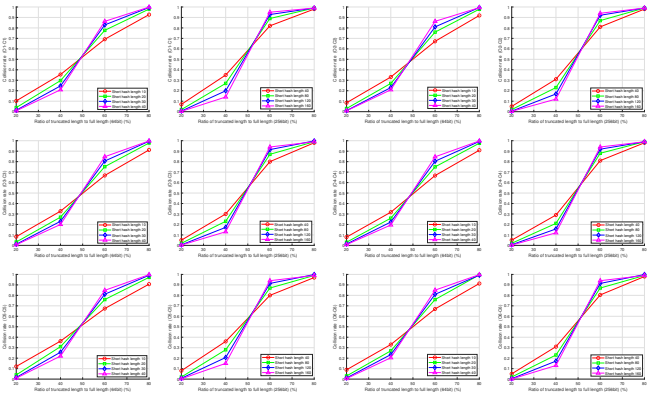

Fig. 12. Short Hash Collision Probability Evaluation (left: 64-bit; right: 256-bit)

**The superiority of ER compared to BP**: In the similarity verification process, the cloud server needs to process the tags from the client and blindly compare them with the tags' in the database. While traditional methods usually use Bilinear Pairing (BP), this scheme proposes a novel blinding method based on Exchange Rule (ER). As shown in Fig. 13, the experimental results show that the ER method is 168 times faster than BP under 64-bit conditions, and the performance advantage is further extended to 390 times under 256-bit conditions. This significant performance improvement stems from the fact that the ER method only needs to perform the XoR operation, which has an essential efficiency advantage over the complex computational process of BP.

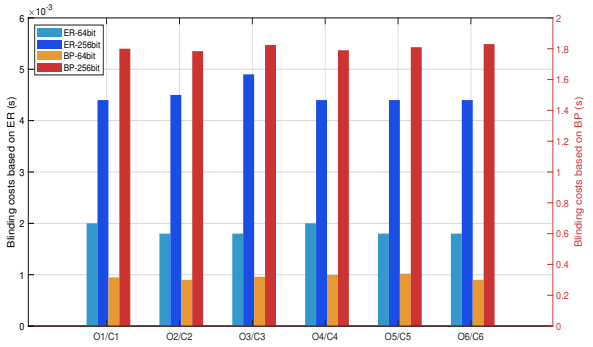


Fig. 13. Parameter Blinded Evaluation (ER: exchange rule, BP: bilinear pair)

**Evaluation of error deduplication rates:** Fig. 14 provides the probability of data being deleted by mistake with different threshold selections. The results show that under 64bit, when the threshold value is 9, its false deletion rate is about 0.25%, and under 256bit, when the threshold value is 1/3 of the fingerprint length, its false deletion rate is not more than 1.2% . These results indicate that the probability of data being deleted by mistake is very low in the data of this scheme.

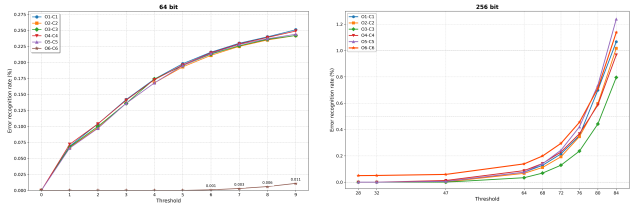


Fig. 14. False Deletion Rates on Different Datasets (left 64bit, right 256 bit)

**Evaluation of security and efficiency of tampering:** Fig. 15 evaluates the probability of an adversary successfully tampering with the data compared to the theoretical value, and shows that the probability of an adversary's successful stringing is much lower than the theoretical value, which suggests that an adversary could not successfully tamper with the data. Fig. 16 gives, the success rate of tampering detection under different parameter settings, and the results show that the scheme is able to detect malicious tampering with a success rate of not less than 95%. Fig. 17 gives that even with large parameter settings, the time overhead required for its required anti-tamper attack does not exceed 0.1 s. Fig. 18 gives the time overhead required for tamper detection at different security levels. Fig. 19 gives the time overhead of tamper detection versus the coefficient R of the encoding.

**Comparison of storage optimization in related work:** This experiment compares the performance of four representative deduplication methods with our work on six datasets ($O_1/C_1$-$O_6/C_6$). By adjusting the thresholds (1-3) and tag lengths (64/256bit), the experiment finds that: Chen *et al.'s* [15]'s method based on SHA256 fingerprints has the lowest deduplication rate due to avalanche effect; Jiang *et al.'s* [18] and Cheng *et al.'s* [19]'s scheme significantly degrades its

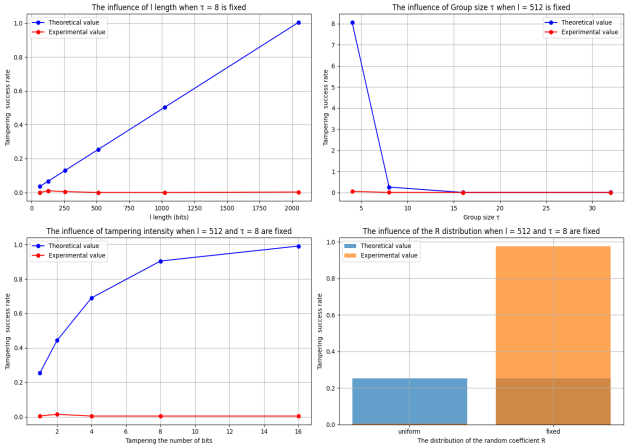


Fig. 15. Probability of Successful Parameter Tampering by Adversary

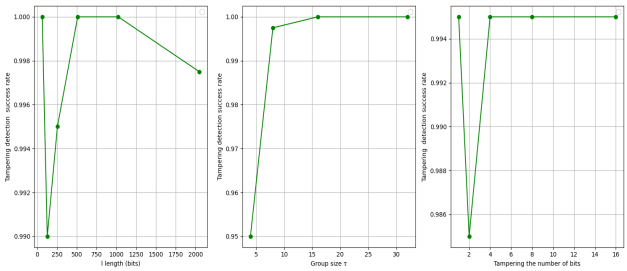


Fig. 16. Probability of Successful Tamper Detection

performance on the $O_6$-$C_6$ dataset due to ignoring the color features. Tang *et al.'s* [20]'s method, although similar to the performance of this study (difference $< 5\%$), lacks the non-interactive key transfer and tamper detection features.

**Comparison of computational costs of related work:** Table III and Fig. 21 gives a comparative analysis of related work in terms of computational overhead. Specifically, this scheme consumes $2FE$ more than Cheng *et al.'s* [19] in the *T/K-Gen* phase, which stems from the non-interactive key transfer mechanism that we adopt, which, although slightly increasing the computational overhead, achieves a constant amount of communication and avoids the needs for the user to stay online ( In [19]) the number of communication rounds increases with the number of participants, and the participants must stay online). In the *Sec-Ver* phase, we have more $H + AMD$ than Cheng *et al.'s* [19] because our scheme achieves robust

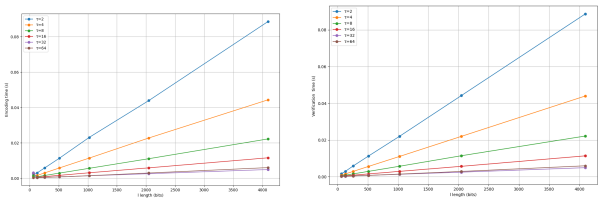

Fig. 17. Encoding and Validation overhead Required for Tamper Detection.

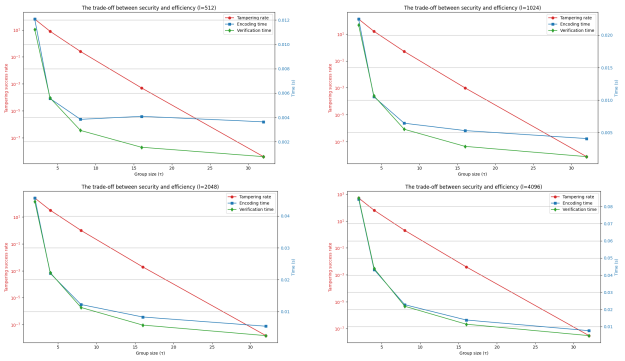


Fig. 18. The Trade-off between Security and Efficiency

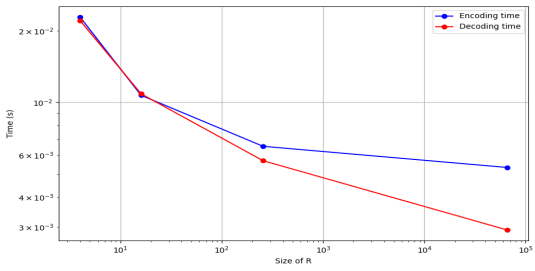


Fig. 19. Effect of Coefficient R on Time Required for Encoding and Decoding

validation of the open parameters, which [19] does not have.

## VI. Conclusion

This paper focuses on the storage and security of outsourced data. Data similarity is defined based on information entropy

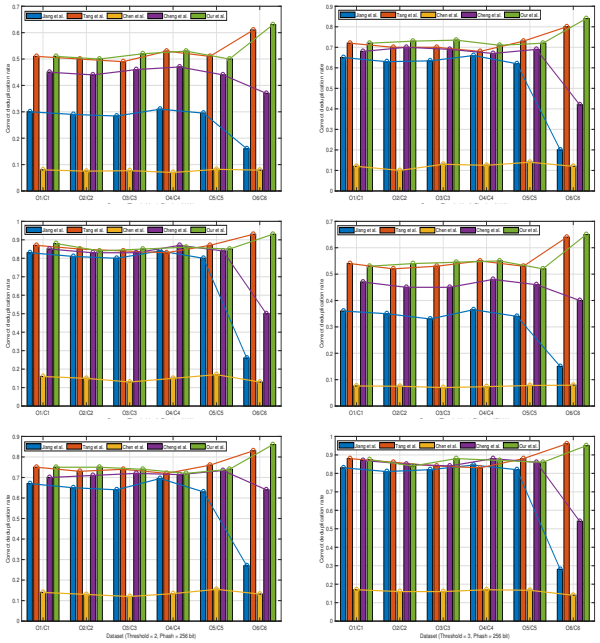


Fig. 20. Comparison of Deduplication Rates of Related Works

theory. Furthermore, the improved fuzzy extractor not only enables non-interactive key transfer but also achieves secure data deduplication without requiring online client assistance. Extensive experiments demonstrate that this approach offers advantages in both security and storage efficiency.

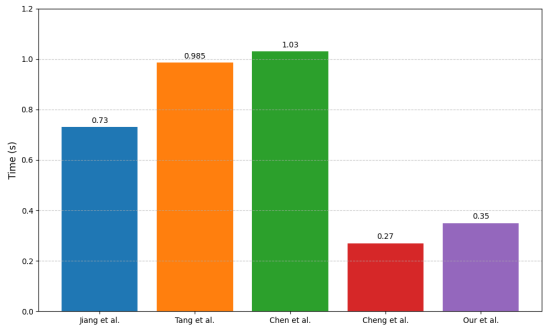


Fig. 21. Comparison of Computational Overhead for Related Works

## REFERENCES


[1] J. McLeod and B. Gormly, "Records storage in the cloud: are we modelling the cost?," *Archives and Manuscripts*, vol. 46, no. 2, pp. 174–192, 2018.

[2] D. T. Meyer and W. J. Bolosky, "A study of practical deduplication," *ACM Transactions on Storage (ToS)*, vol. 7, no. 4, pp. 1–20, 2012.

[3] G. Wallace, F. Douglis, H. Qian, P. Shilane, S. Smaldone, M. Chamness, and W. Hsu, "Characteristics of backup workloads in production systems.," in *FAST*, vol. 12, pp. 4–4, 2012.

[4] J. Li, P. P. Lee, Y. Ren, and X. Zhang, "Metadedup: Deduplicating metadata in encrypted deduplication via indirection," in *2019 35th Symposium on Mass Storage Systems and Technologies (MSST)*, pp. 269–281, IEEE, 2019.

[5] M. E. Smid and D. K. Branstad, "Data encryption standard: past and future," *Proceedings of the IEEE*, vol. 76, no. 5, pp. 550–559, 1988.

[6] F. A. Petitcolas, R. J. Anderson, and M. G. Kuhn, "Information hiding-a survey," *Proceedings of the IEEE*, vol. 87, no. 7, pp. 1062–1078, 1999.

[7] M. Bellare, S. Keelveedhi, and T. Ristenpart, "Message-locked encryption and secure deduplication," in *Annual international conference on the theory and applications of cryptographic techniques*, pp. 296–312, Springer, 2013.

[8] T. Jiang, X. Chen, Q. Wu, J. Ma, W. Susilo, and W. Lou, "Secure and efficient cloud data deduplication with randomized tag," *IEEE transactions on information forensics and security*, vol. 12, no. 3, pp. 532–543, 2016.

[9] Z. Yan, L. Zhang, D. Wenxiu, and Q. Zheng, "Heterogeneous data storage management with deduplication in cloud computing," *IEEE Transactions on Big Data*, vol. 5, no. 3, pp. 393–407, 2017.

[10] Y. Fu, N. Xiao, T. Chen, and J. Wang, "Fog-to-multicloud cooperative ehealth data management with application-aware secure deduplication," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 5, pp. 3136–3148, 2021.

[11] M. Song, Z. Hua, Y. Zheng, H. Huang, and X. Jia, "Blockchain-based deduplication and integrity auditing over encrypted cloud storage," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 6, pp. 4928–4945, 2023.

[12] Y. Shin, D. Koo, J. Yun, and J. Hur, "Decentralized server-aided encryption for secure deduplication in cloud storage," *IEEE Transactions on Services Computing*, vol. 13, no. 6, pp. 1021–1033, 2017.

[13] H. Zheng, S. Zeng, H. Li, and Z. Li, "Secure batch deduplication without dual servers in backup system," *IEEE Transactions on Dependable and Secure Computing*, 2024.

[14] J. R. Douceur, A. Adya, W. J. Bolosky, P. Simon, and M. Theimer, "Reclaiming space from duplicate files in a serverless distributed file system," in *Proceedings 22nd international conference on distributed computing systems*, pp. 617–624, IEEE, 2002.

[15] R. Chen, Y. Mu, G. Yang, and F. Guo, "Bl-mle: Block-level message-locked encryption for secure large file deduplication," *IEEE Transactions on Information Forensics and Security*, vol. 10, no. 12, pp. 2643–2652, 2015.

[16] X. Li, J. Li, and F. Huang, "A secure cloud storage system supporting privacy-preserving fuzzy deduplication," *Soft Computing*, vol. 20, pp. 1437–1448, 2016.

[17] J. Takeshita, R. Karl, and T. Jung, "Secure single-server nearly-identical image deduplication," in *2020 29th International Conference on Computer Communications and Networks (ICCCN)*, pp. 1–6, IEEE, 2020.

[18] T. Jiang, X. Yuan, Y. Chen, K. Cheng, L. Wang, X. Chen, and J. Ma, "Fuzzydedup: Secure fuzzy deduplication for cloud storage," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 3, pp. 2466–2483, 2022.

[19] S. Cheng, Z. Tang, S. Zeng, X. Cui, and T. Li, "Pfdup: Practical fuzzy deduplication for encrypted multimedia data," *Journal of Industrial Information Integration*, vol. 40, p. 100613, 2024.

[20] Z. Tang, S. Zeng, S. Han, Y. Feng, T. Li, and M. He, "Fuzzy deduplication: Color-aware deduplication for multi-media data," *IEEE Transactions on Services Computing*, 2024.

[21] M. Song, Z. Hua, Y. Zheng, T. Xiang, and X. Jia, "Simless: A secure deduplication system over similar data in cloud media sharing," *IEEE Transactions on Information Forensics and Security*, 2024.

[22] Y. Dodis, L. Reyzin, and A. Smith, "Fuzzy extractors: How to generate strong keys from biometrics and other noisy data," in *Advances In Cryptology-EUROCRYPT 2004: International Conference On The Theory And Applications Of Cryptographic Techniques, Interlaken, Switzerland, May 2-6, 2004. Proceedings 23*, pp. 523–540, Springer, 2004.

[23] R. Cramer, Y. Dodis, S. Fehr, C. Padró, and D. Wichs, "Detection of algebraic manipulation with applications to robust secret sharing and fuzzy extractors," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pp. 471–488, Springer, 2008.

[24] J. Dave, A. Dutta, P. Faruki, V. Laxmi, and M. S. Gaur, "Secure proof of ownership using merkle tree for deduplicated storage," *Automatic Control and Computer Sciences*, vol. 54, pp. 358–370, 2020.

[25] K. Huang, X. Zhang, Y. Mu, F. Rezaeibagha, and X. Du, "Bidirectional and malleable proof-of-ownership for large file in cloud storage," *IEEE Transactions on Cloud Computing*, vol. 10, no. 4, pp. 2351–2365, 2021.

[26] S. Keelveedhi, M. Bellare, and T. Ristenpart, "{DupLESS}:{Server-Aided} encryption for deduplicated storage," in *22nd USENIX security symposium (USENIX security 13)*, pp. 179–194, 2013.

[27] J. Liu, N. Asokan, and B. Pinkas, "Secure deduplication of encrypted data without additional independent servers," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, pp. 874–885, 2015.

[28] G. Ha, C. Jia, Y. Huang, H. Chen, R. Li, and Q. Jia, "Scalable and popularity-based secure deduplication schemes with fully random tags," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 3, pp. 1484–1500, 2023.

[29] L. Chen, F. Xiang, and Z. Sun, "secure sinsed on hashing and clustering in cloud storage.," *KSII Transactions on Internet & Information Systems*, vol. 15, no. 4, 2021.

[30] Y. Gao, L. Chen, J. Lai, T. Wang, X. Wu, and S. Yu, "Iot-dedup: Device relationship-based iot data deduplication scheme," *IEEE Transactions on Parallel and Distributed Systems*, 2025.

[31] Y. Zhou, D. Feng, Y. Hua, W. Xia, M. Fu, F. Huang, and Y. Zhang, "A similarity-aware encrypted deduplication scheme with flexible access control in the cloud," *Future Generation Computer Systems*, vol. 84, pp. 177–189, 2018.

TABLE IV
IMPORTANT NOTATIONS

| Notation | Description | Parameter | Decision Variable |
|---|---|---|---|
| $ex_C$ | exchange factor of C | - | - |
| $ex_S$ | exchange factor of S | - | - |
| $ex_{SC}$ | exchange factor of SC | - | - |
| $M/F$ | user's files | - | - |
| $C_{M/F}$ | encrypted file of $M/F$ | $M/F$ and encryption key$K_w$ | - |
| $Phash$ | perceived hash of $M/F$ | text, images, video, audio | datatype |
| $sh$ | short hash of $Phash$ | $Phash$ | hash length |
| $combine(\bullet)$ | r andomized combinations of $ex$ | pair of $ex$ | ($ex_C$, $ex_{CS}$, $ex_S$) |
| $d(\bullet)$ | Hanming distance | $Phash_1, Phash_2$ | - |
| $PP/P$ | public parameters of FE | $M/F$ and random number | - |
| $K_w$ | FE generated key | $M/F$ and random number | - |
| $fk$ | PoW authentication Key | random number and $Phash$ | - |
| $R_{out}$ | AMD's private code | - | - |
| $R_{mac}$ | AMD's public code | - | - |
| $\tau_1$ | similarity determination threshold for $sh$ | - | - |
| $\tau_2$ | similarity Determination Threshold for $Phash$ | - | - |
| $SE^{+/-}_{k_w}(\bullet)$ | symmetric encryption algorithm | $M/F$ | $k_w$, + encryption, - decryption |
| $\oplus$ | XoR operation | pair of binary strings | - |